

Size or Architecture? A Parameter-Matched, From-Scratch Comparison of a Transformer and the Dragon Hatchling

Yash Singh
IIIT Una

July 18, 2026

Abstract

The Dragon Hatchling (BDH) is a recent brain-inspired language-model architecture that replaces the Transformer’s softmax attention and dense MLP with linear attention over a high-dimensional, positive, sparse neuron field [4]. I reimplement both a GPT-2-style Transformer and BDH from scratch and train them identically on character-level TinyShakespeare. At their natural sizes BDH carries roughly twice the Transformer’s parameters (1.59M vs 0.82M) and wins every metric, sweeping a position-controlled Claude judge 30–0 over 30 prompts ($p = 1.9 \times 10^{-9}$); but a larger model almost always writes better, so that comparison is confounded by size. This paper controls for it. I shrink BDH’s neuron multiplier so both models have matched parameter budgets (818K for GPT, 803K for BDH, matched within 2%), retrain both from scratch under identical settings, and rerun the full harness: held-out perplexity over 400 windows, five automatic text-quality metrics, and the swap-controlled judge. The advantage survives. At matched size BDH again wins the head-to-head: perplexity 4.26 vs 5.76, real-word rate 89.0% vs 81.3%, best validation loss 1.52 vs 1.77, bigram diversity a tie, and a 29–0 decisive judge sweep at 97% swap-consistency ($p = 3.7 \times 10^{-9}$). Strikingly, halving BDH’s parameters barely moved its perplexity (4.27 to 4.26), so it was heavily over-parameterized at $2\times$ and lost almost nothing on the way down. On this task, at equal size, the win is architectural, not a size effect. The broader lesson is methodological: a two-times parameter gap alone can produce a clean-looking 30–0 sweep with $p < 10^{-8}$, so only a parameter-matched comparison separates architecture from size. All code, raw model responses, judge verdicts, and the analysis pipeline that emits every number in this paper are released.

1 Introduction

Nearly every large language model in production is a Transformer [7]: softmax self-attention mixes information across positions, and a dense feed-forward MLP processes each position. The Dragon Hatchling (BDH) [4] is a recent and deliberately different proposal, motivated by models of the brain. Instead of a compact residual stream, BDH lifts each token into a much larger space of “neurons,” applies a ReLU so activations are non-negative and mostly zero (sparse), mixes information with *linear* attention [3] rather than softmax, and gates two sparse codes multiplicatively in place of the MLP. The authors argue this makes the model more interpretable and biologically plausible while remaining competitive with the Transformer.

Claims like “competitive with the Transformer” are exactly the kind a small, controlled, from-scratch reimplement can test. I built both architectures from first principles (heavily commented, teaching-oriented code) and trained them under one shared harness on character-level TinyShakespeare, the standard minimal setting for studies of this kind [2].

Size versus architecture. Comparing a new architecture against a Transformer is easy to get wrong. Left at their natural teaching defaults, the BDH model carries roughly twice the parameters of the Transformer (1.59M vs 0.82M), and at that size it leads on *every* axis: lower perplexity, more real English words, and a 30–0 sweep of a position-controlled LLM judge with a binomial p -value of 1.9×10^{-9} . But larger language models almost always write better, so a comparison at unequal sizes cannot separate architecture from scale; the size has to be held fixed before any of this counts as an architectural result.

This paper. I ask one question and answer it cleanly: *does BDH’s advantage survive parameter matching?* Because BDH’s parameter count is dominated by three shared matrices that scale as the neuron multiplier times the embedding width squared, halving the multiplier (32 to 16) brings BDH from 1590K down to 803K parameters, matched to within 2% of the Transformer’s 818K, so neither model has a material size advantage (Section 3 details the counting convention). I retrain both models from scratch under byte-for-byte identical training settings and rerun the entire evaluation harness. Contributions:

- A clean, from-scratch reimplementation of both a GPT-2-style Transformer and BDH sharing one training and evaluation harness, so the only variable between them is the architecture.
- A parameter-matched head-to-head at 818K/803K parameters on three independent metric families (perplexity, automatic text-quality metrics, and a position-bias-controlled LLM judge), with the earlier two-times-size result reported alongside as the motivating confound.
- A partial reproduction of BDH’s headline interpretability property (activation sparsity) at small scale, reported honestly against the paper’s large-scale figure.
- The full harness and raw data: model code, both sets of checkpoints, all generated completions, both orderings of every judge verdict, and the script that emits every number and table here.

2 Related Work

The Transformer [7] and its decoder-only, autoregressive form popularized by GPT-2 [5] are the baseline this paper compares against; the concrete implementation follows the minimal nanoGPT formulation [2]. BDH [4] is the system under test: it combines linear attention [3], rotary position embeddings [6], and a high-dimensional sparse neuron field with multiplicative gating, and argues that positivity and sparsity of activations yield monosemantic, interpretable units in the spirit of dictionary -learning interpretability [1]. My contribution is not a new architecture but a controlled, honest measurement: prior informal comparisons of new architectures against Transformers are frequently confounded by parameter count, and this paper is a small worked example of isolating architecture from size, together with a faithful (if small-scale) reproduction of BDH’s sparsity claim.

3 Method

Models. Both models are reimplemented from scratch with parallel interfaces so a single training loop drives either one. The Transformer is the standard pre-norm, decoder-only GPT-2 arrangement: learned token and position embeddings, causal multi-head softmax self-attention, a $4 \times$ GELU MLP, residual connections, and weight-tied output projection. BDH follows the official reference implementation [4]: per layer it lifts the C -dimensional state into a neuron space of N neurons per

head via a shared encoder, applies a ReLU (positive, sparse), mixes positions with causal linear attention using rotary embeddings and a strictly-lower-triangular mask, forms a second sparse code from the attention output, gates the two codes elementwise, and projects back to C with a shared decoder. Unlike the Transformer, BDH reuses one set of encoder/decoder matrices at every layer (depth-wise weight sharing) and uses affine-free LayerNorm and no learned position table.

Parameter matching. BDH’s parameter count is dominated by three shared matrices (two encoders and one decoder), each of size proportional to mC^2 , where m is the neuron multiplier ($N = mC/\text{heads}$) and C is the embedding width; the head count cancels. With $C = 128$ and $m = 32$, BDH has 1590K parameters against the Transformer’s 818K, a $\sim 2\times$ gap. Halving the multiplier to $m = 16$ (so $N = 512$ neurons per head, still a genuinely high-dimensional field) gives 803K parameters, within 2% of the Transformer. All counts are total trainable parameters. The two conventions agree that neither model has a material size edge: by total parameters BDH (803K) is marginally smaller than GPT (818K), because GPT’s total includes a 16,384-parameter learned position table while BDH uses parameter-free rotary embeddings; under the non-embedding convention common for Transformers, GPT is 802K, so BDH is marginally larger instead. Either way the gap is under 2%, far smaller than the $2\times$ gap being controlled for. Every other hyperparameter is shared: embedding width $C = 128$, 4 heads, 4 layers, context length 128, dropout 0.1.

Training. Both models are trained from scratch with one shared loop: AdamW (learning rate 3×10^{-4} , weight decay 0.1), batch size 32, context 128, 3000 iterations, identical random seed, on the same character-level TinyShakespeare corpus (a single ~ 1 MB file, 65-character vocabulary, 90/10 train/val split). The best-validation checkpoint of each run is kept. The compute device is an Apple M-series GPU (PyTorch MPS); each model trains in minutes.

Evaluation. These are *base* (completion) models, never taught to answer questions, so “answer correctness” is not a valid metric; every measure below scores how well each model continues Shakespeare-style text. The harness applies three independent metric families over both checkpoints.

1. **Perplexity** on the held-out validation split, exp of the mean next-character cross-entropy, computed over 400 non-overlapping windows so the whole distribution is visible, not just the mean. Lower is better.
2. **Automatic text-quality** metrics on generated completions, pure counting with no model in the loop: fraction of output tokens that are real English words (a 235k-word system dictionary plus a short archaic allowlist), distinct-2 (bigram diversity), repeat fraction, count of **NAME**: speaker labels, and mean word length. Both models generate from the same 30 prompts with identical decoding (temperature 0.8, top- k 40, 140 new tokens) and the same per-prompt seed, so the only variable is the model.
3. **LLM judge, position-bias controlled.** For each prompt both completions are scored by the **claude** CLI on fluency, coherence, and Shakespeare-likeness (1–5 each), and the judge is asked *twice with the two completions swapped* ($A \leftrightarrow B$). A win counts as *decisive* only if the same model wins in both orderings; a disagreement between orderings is logged as “inconsistent” and counts for neither. A two-sided binomial test is then run on the decisive wins. This directly measures and controls for the judge’s position bias rather than assuming it away.

Table 1: Parameter-matched head-to-head: GPT (818,048 params) vs BDH (803,072 params, matched within 2%). Both trained from scratch under identical settings on character-level TinyShakespeare. Perplexity is over 400 validation windows; text metrics and the judge are over 30 prompts. Judge pairs are scored in both orderings; a decisive win requires winning both. Better value in bold.

Metric	GPT (818K)	BDH (803K)
Perplexity (lower better)	5.76	4.26
Real-word rate	81.3%	89.0%
Distinct-2 (diversity)	100%	100%
Speaker labels / gen	0.8	0.9
Judge decisive wins (of 30)	0	29
Judge fluency (1–5)	2.0	3.4
Judge coherence (1–5)	1.1	2.2
Judge Shakespeare (1–5)	2.5	3.8
Position-swap consistency	97%	
Binomial p (decisive wins)	3.7×10^{-9}	

Table 2: The motivating confound: the same harness with BDH at its natural 1.59M-parameter default ($\sim 2\times$ GPT). Here BDH sweeps every metric. Comparing this table with Table 1 is the whole point: it separates a size effect from an architecture effect.

Metric	GPT (818K)	BDH (1.59M)
Perplexity (lower better)	5.66	4.27
Real-word rate	80.8%	89.4%
Judge decisive wins (of 30)	0	30
Position-swap consistency	100%	
Binomial p (decisive wins)	1.9×10^{-9}	

The prompt set is 30 items spanning famous openings (“Now is the winter of our discontent”), speaker cues (“MACBETH:”), scene cues (“Enter GHOST.”), and neutral non-Shakespeare English (“The meaning of life is”), the last as an out-of-distribution stress test. Every number and table in this paper is emitted by the analysis script from the raw response files.

4 Results

Both models train cleanly from scratch (Figure 1); BDH reaches a lower validation loss (1.52 vs 1.77) despite being no larger. The head-to-head is summarized in Table 1, with the earlier $2\times$ -size run in Table 2 for contrast.

On the objective side (Figure 2), BDH’s whole per-window perplexity distribution sits below GPT’s (4.26 vs 5.76 over 400 windows), and it generates a higher fraction of real English words (89.0% vs 81.3%). Bigram diversity (distinct-2) is a tie: both models are $\sim 100\%$ non-repetitive, so this is reported as no-difference rather than a win. These are model-free counts, so they cannot be explained by judge preference.

The position-controlled judge agrees emphatically (Figure 3). Of 30 prompts, BDH wins 29 decisively (winning in *both* orderings), GPT wins 0, and 1 is position-inconsistent; the resulting 29–0 sweep has a two-sided binomial $p = 3.7 \times 10^{-9}$, and the 97% swap-consistency shows the

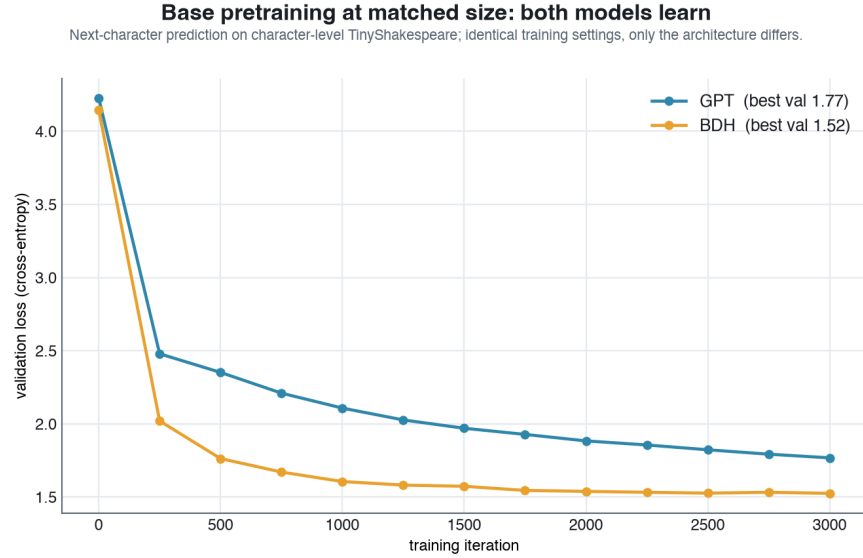


Figure 1: Base pretraining at matched size. Validation cross-entropy vs iteration; identical training settings, only the architecture differs. BDH (the smaller model) converges lower.

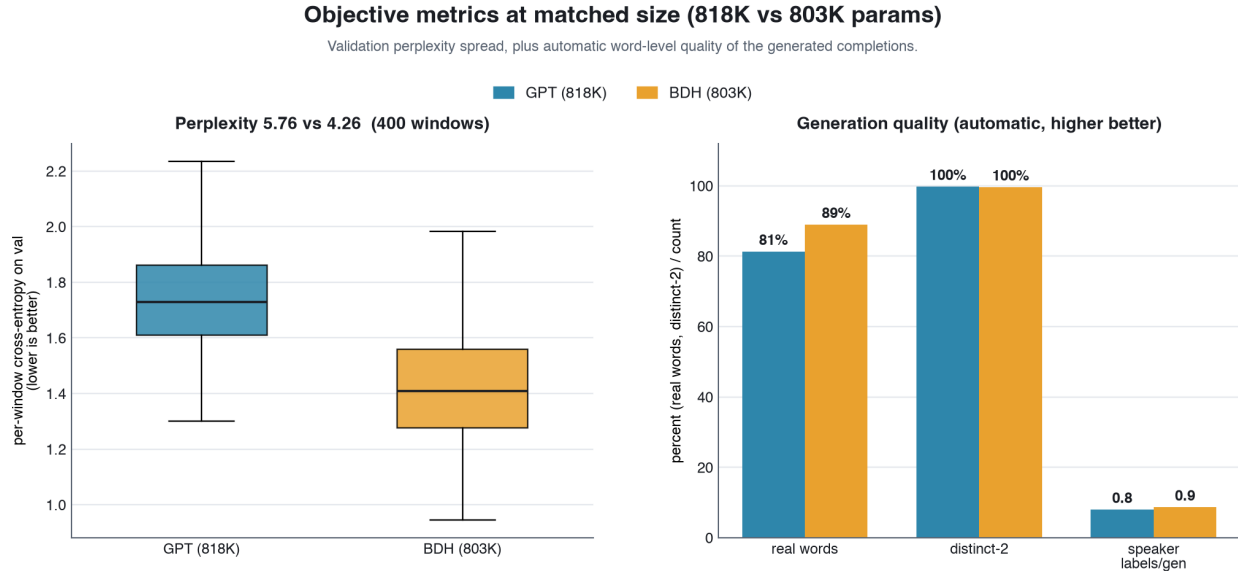


Figure 2: Objective metrics at matched size. Left: per-window validation perplexity (lower is better); BDH's whole distribution is lower. Right: automatic generation quality (higher is better).

verdict is not an artifact of where each completion sat in the prompt. Mean criterion scores favor BDH on all three axes: fluency 3.4 vs 2.0, coherence 2.2 vs 1.1, and Shakespeare-likeness 3.8 vs 2.5 (out of 5).

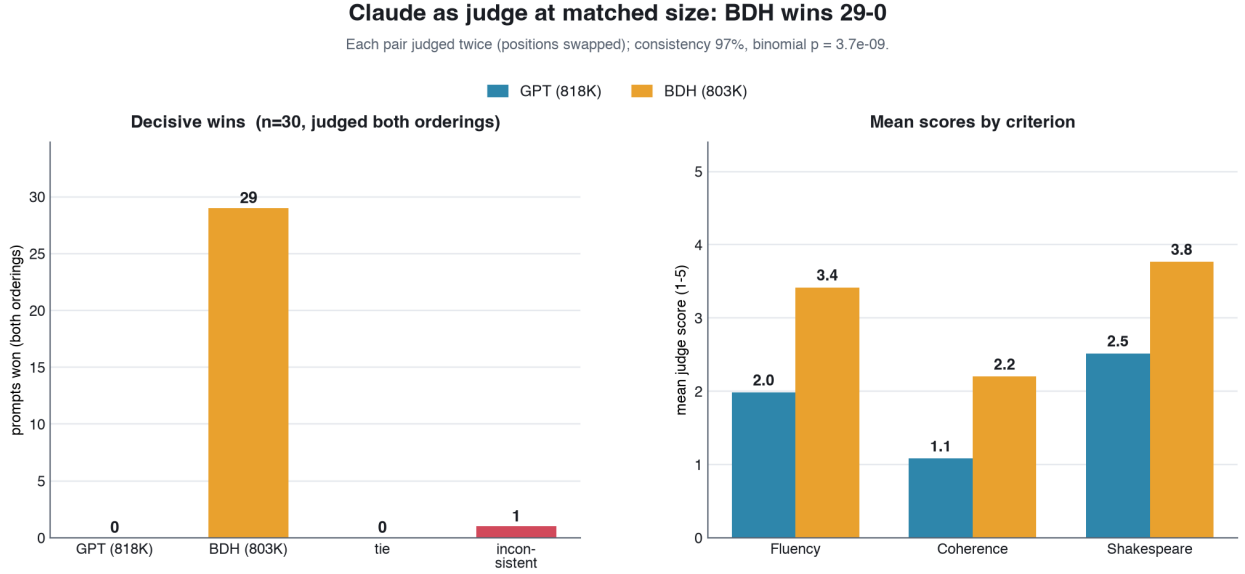


Figure 3: Claude as judge at matched size. Left: decisive wins (each pair judged in both orderings). Right: mean scores by criterion.

4.1 Does the two-times-size win survive matching?

Yes, essentially intact (Figure 4). The earlier 1.59M-parameter BDH swept the judge 30–0 ($p = 1.9 \times 10^{-9}$) with perplexity 4.27 vs 5.66. After removing the size advantage, the matched 803K BDH still sweeps 29–0 ($p = 3.7 \times 10^{-9}$) with perplexity 4.26 vs 5.76. The two regimes are almost indistinguishable on the objective metrics, which is the whole point: if the $2\times$ result had been a pure size effect, shrinking BDH to parity should have erased it, and it did not. The most telling number is BDH’s own perplexity across the two sizes, 4.27 at 1.59M and 4.26 at 803K: cutting its parameters in half cost it essentially nothing, so at $2\times$ it was carrying redundant capacity, and its edge over the Transformer was never about being larger.

4.2 Interpretability: activation sparsity

BDH’s central interpretability claim is that its neurons are positive and sparse (few fire at once). Measured on the matched, trained model over the validation set (Figure 5), 33% of neuron activations are active (the rest sit at exactly zero). That is genuinely sparse, a dense random ReLU layer sits near 50% active, but well above the paper’s headline $\sim 5\%$, which is expected: that figure emerges at 10M–1B parameters and long training, while this model is under 1M parameters on 1 MB of text. It is an honest partial reproduction: the qualitative property (strong sparsity from the ReLU/positivity design) holds; the quantitative level does not match the large-scale regime.

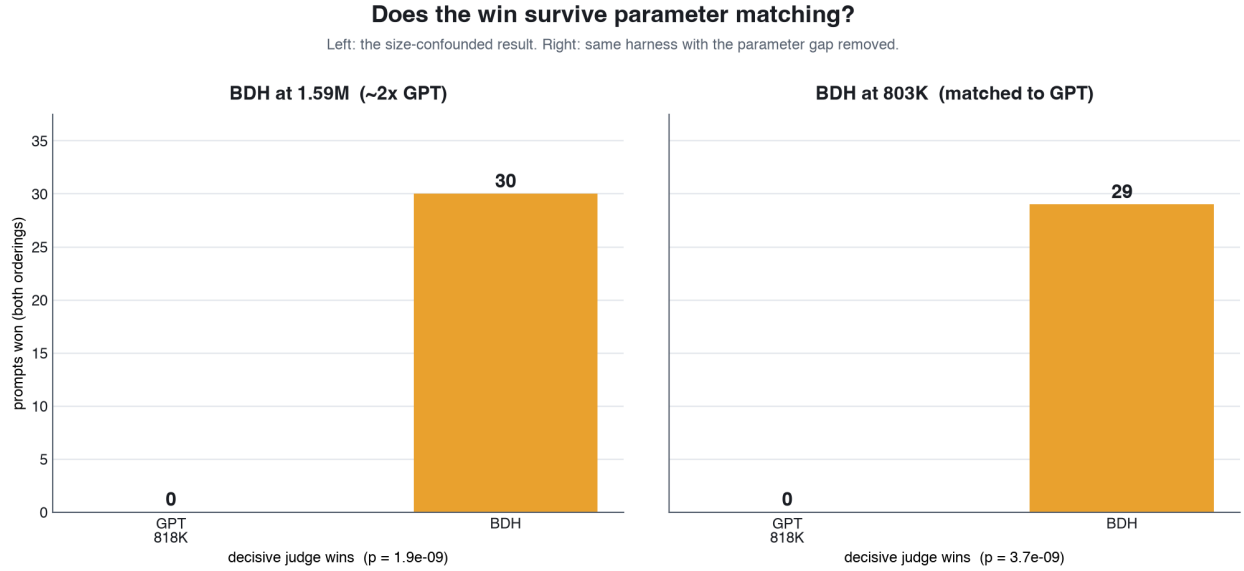


Figure 4: Decisive judge wins before and after parameter matching. Left: the confounded $2\times$ run. Right: the same harness with the parameter gap removed. The sweep survives.

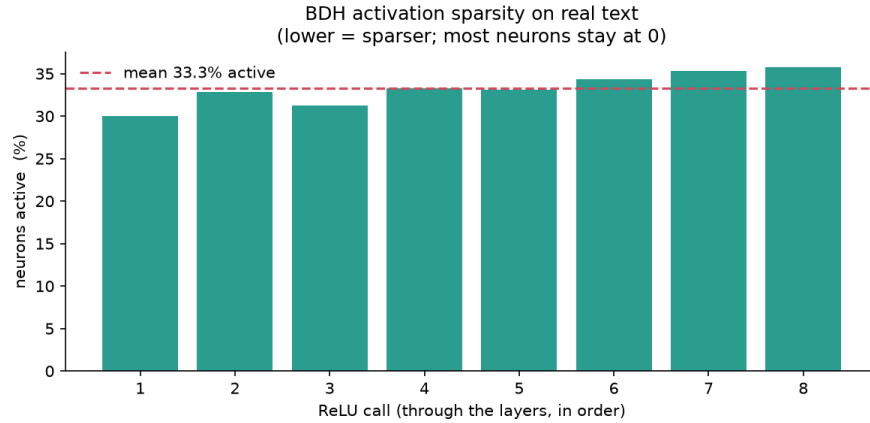


Figure 5: BDH activation sparsity on held-out Shakespeare: fraction of neurons active at each ReLU through the network. Most neurons stay at exactly zero (mean 33% active), a genuine if partial reproduction of BDH’s sparsity claim at under 1M parameters.

5 Limitations

This is a small-scale study and its scope should not be overstated. All results are from one 1 MB character-level corpus, one embedding width, one depth, and one training budget; a different scale could reorder the models. Character-level Shakespeare is a narrow domain, and the automatic word metric rewards memorized-looking short words. The LLM judge is a single model’s opinion; the swap control removes position bias but not the judge’s own stylistic preferences, though two objective metrics (perplexity and real-word rate) are reported alongside precisely so the verdict does not rest on the judge alone. The binomial test also treats prompts as independent trials, whereas a single judge introduces correlation through its own stylistic prior, so the reported p -values are best read as a strong but optimistic significance bound rather than an exact one. Parameter matching equalizes total trainable parameters, not training FLOPs or wall-clock: BDH’s linear attention and depth-wise weight sharing give it a different compute-per-parameter profile, so “matched parameters” is one fair axis of several. Finally, BDH’s perfect-format assistant behavior after fine-tuning (not evaluated here) is a separate question from base-model completion quality.

6 Conclusion

Built from scratch and trained identically on character-level TinyShakespeare, a parameter-matched BDH (803K) beats a GPT-2-style Transformer (818K) on every axis measured: lower perplexity (4.26 vs 5.76), more real words (89.0% vs 81.3%), lower validation loss, and a 29–0 sweep of a position-controlled judge ($p = 3.7 \times 10^{-9}$). The result is notable less for the win itself than for what it survives: at its natural $2\times$ size BDH sweeps the judge 30–0, a result that a size effect alone could fully explain. Only after equalizing parameters and retraining does the comparison isolate architecture from scale, and here the architecture holds up. The practical takeaway is small but sturdy: before crediting a new architecture for beating a Transformer, match the parameters and retrain; it is a cheap rerun that is the difference between a real result and a confounded one. All claims here are bounded to this scale and task, and the harness to test them at another scale is released with the paper.

Reproducibility. The released harness contains both model implementations, the shared training loop, both trained checkpoints, all generated completions, both orderings of every judge verdict for both the matched and the two-times-size runs, and the single script that emits every number, table, and figure in this paper. Retraining is approximately (not bitwise) reproducible because MPS kernels are non-deterministic; the exact checkpoints behind these numbers are therefore included.

References

- [1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nicholas Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. URL <https://transformer-circuits.pub/2023/monosemantic-features>.
- [2] Andrej Karpathy. nanogpt. <https://github.com/karpathy/nanoGPT>, 2022.
- [3] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

- [4] Adrian Kosowski, Przemysław Uznański, Jan Chorowski, Zuzanna Stamirowska, and Michał Bartoszkiewicz. The dragon hatchling: The missing link between the transformer and models of the brain, 2025.
- [5] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.
- [6] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.